

Module 23: Object Lifetime, Destructors & RAII

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

This module teaches students exactly when objects are born and when they die, and turns that knowledge into the single most important resource-management idea in C++: RAII. Students trace constructors and destructors to build an accurate mental model of automatic, dynamic and static lifetime, learn the reverse-order rule for members, bases and locals, and then see RAII guarantee clean-up on every exit path - including early returns, and (in Module 31) exceptions. The module closes on a deliberate cliffhanger: an RAII class that owns heap memory breaks when copied, which Module 24 resolves.

Suggested Duration: 75–90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Distinguish automatic, dynamic and static lifetime, and predict when each kind of object is destroyed.
- Write a destructor and explain when it runs automatically.
- Predict the order of construction and destruction for locals, members, and base/derived classes.
- Explain RAII: acquiring a resource in a constructor and releasing it in a destructor.
- Use an RAII class to guarantee clean-up on every exit path of a function.

Prerequisites

- Module 22 (multi-file projects) - this module's project is a CMake project.
- Part 2 Module 10 (new/delete) and Module 12 (virtual destructors).

Materials & Instructor Prep

- 23-object-lifetime-raii/examples/01_tracing_lifetime.cpp and 02_construction_order.cpp - run them only AFTER students write predictions.
- Paper for written output predictions.

Key Vocabulary

Term	Definition
Automatic lifetime	An object declared in a block lives until the end of that block and is destroyed automatically on every way out of it.

Term	Definition
Dynamic lifetime	An object created with new lives until delete is called on it - or forever, if delete is forgotten.
Destructor	A special member function, ~ClassName(), that runs automatically when an object dies.
Stack unwinding	Leaving blocks and functions in turn while destroying their local objects - on return, and when an exception propagates.
Temporary	An unnamed object that lives until the end of the full expression that created it.
RAII	Resource Acquisition Is Initialization: a resource is acquired in a constructor and released in the destructor, so its lifetime is tied to an object's scope.

Lesson Outline & Instructor Talking Points

1. Predict, then trace (20 min)

- Before running examples/01_tracing_lifetime.cpp, have every student write down the full output they expect. Then run it and compare line by line.
- Draw attention to the temporary Sensor("Temp") dying on the same line - most predictions get this wrong.

Instructor note: *The prediction step is the lesson. Students who just watch the output learn far less than students who discover their own model was wrong.*

2. Construction and destruction order (15 min)

- Run examples/02_construction_order.cpp after predictions. State the one rule: destruction is always the reverse of construction.
- Ask why reverse order makes sense: later parts may depend on earlier ones, and in the destructor body every member is still alive.

3. Heap objects and leaks (10 min)

- Run examples/03_heap_lifetime.cpp and point out the missing [-] line for the leaked sensor. Emphasise that a leak isn't just wasted memory - every bit of clean-up in that destructor never happens.

4. RAII (25 min)

- Walk through examples/05_raii_early_return.cpp. Ask students to find the bug in driveManual() before you point to it.
- Show the RAII version: there is nothing to remember, so nothing to forget. Then run examples/04_raii_logger.cpp and connect it to std::ofstream closing itself in Module 14.
- List the RAII classes students already use: std::vector, std::string, file streams.

Instructor note: *This is the payoff of the module and the foundation of Modules 24-26 and 31. Give it time, and say plainly: modern C++ code rarely calls delete because of this idea.*

5. The cliffhanger (5 min)

- Show the SampleBuffer class from the README and ask what happens if it is copied. Leave the question open for Module 24.

Common Student Misconceptions

Misconception	How to Address It
"I should call the destructor myself when I'm done with an object."	Destructors run automatically - at the end of scope or inside delete. Calling one yourself makes it run twice.
"A heap object is destroyed when the pointer variable goes out of scope."	Only the pointer dies. The object lives until delete is called - which is why raw new so easily leaks.
"Members are destroyed before the destructor body runs."	The destructor body runs first, while every member is still alive; members are destroyed afterwards, in reverse order.
"RAII is only about memory."	Any resource that must be given back - files, devices, locks, a switched-on motor - can and should be managed with RAII.

Exercises & Module Project

Exercise: 23-object-lifetime-raii/exercises/exercise1.cpp - Part A: a Tracer class and a written output prediction; Part B: an RAII ReadingBuffer that owns a heap array and a processReadings() function with an early return, showing the buffer is freed on both paths. Project (three tracks): Generic - Club Attendance Session that always closes a meeting in the log; EE - a Bench Power Supply Guard that always switches the supply off; Biomedical - an Infusion Pump Session that always stops the pump.

Facilitation guidance: Check that the RAII class does all acquisition in its constructor and all release in its destructor, that main and the scenario functions contain no manual start/stop or open/close calls, and that the tracing output proves release on every early-return path. Students should NOT copy their RAII objects yet - that is Module 24.

Discussion & Comprehension Check Questions

1. Why must an RAII guard be an automatic (stack) object rather than created with new?

Guidance: Only automatic objects are destroyed automatically on every exit path. A guard created with new would need a manual delete - the very thing RAII is meant to remove.

2. Why are members destroyed in reverse order of construction?

Guidance: Later members may depend on earlier ones; destroying in reverse means nothing is destroyed while something that uses it is still alive.

3. Which standard library classes you already use are RAII classes?

Guidance: std::vector and std::string (memory), std::ofstream and std::ifstream (open files) - among others.

Assessment Ideas

- Prediction quiz: a short program with nested blocks, a temporary, and a base/derived pair; students write the exact output.
- Ask students to convert a function with three early returns and manual clean-up into an RAII version.

Answer Key Notes

See `23-object-lifetime-raii/exercises/solutions/exercise1_solution.cpp`, which includes the expected Part A prediction.

Wrap-Up & Bridge to Next Module

Bridge forward: "Our `SampleBuffer` is RAII-perfect - until it is copied, and two objects delete the same array. Next module explains exactly what copying means and how to make it safe."

Instructor Tips

- Never skip the written predictions - they turn passive viewing into active learning.
- If time is short, protect RAII (step 4) over static lifetime, which needs only a mention.